

Collective Intelligence: Ant Colony Simulation & The Traveling Salesman Problem

By Fiona Dubay

Background & Goals:

The motivation for my project came from my interest in collective intelligence. I'm highly intrigued by systems built of simple agents that can solve complex problems; termites and nest construction, slime mold navigation, and ant colony foraging behaviors. Historically, computing systems have been highly anthropocentric– data processing and calculation are done through a centralized system. Centralized processing units could be compared to a human brain. What interested me were systems of an almost opposite build. Specifically, I hoped to create a visualized agent-based simulation modeled off of biological collective systems to solve some kind of problem. As my project developed, the goal became more specifically to create a system of ant agents that could follow simple rules and collectively give a solution for the traveling salesman problem.

The traveling salesman problem (also known as Traveling Sales Person, or TSP) goal involves finding the shortest possible route between a list of given locations where the final stop is the same as the point of origin. In computational theory, TSP is classified as a nondeterministic polynomial (NP) problem. This is a complexity class used to categorize decision problems. Essentially, this means that TSP is a very computationally complex problem, and to be solved would need a brute-force calculation of every possible path length.

The kind of simulation I made falls into a category of other agent-based probabilistic systems called ant colony optimization algorithms (ACOs). Often, ACOs will use a pheromone-based information communication method, modeled on the real-life behavior of ant colonies during foraging. ACOs have been used many times previously as an approach to TSP. In fact, the first ACO – which was called the Ant Colony System, or ACS – was introduced by Marco Dorigo in the 1990s as an approach to TSP.

Methodology:

My approach for this project was a combination of several projects I was able to find online (Goadrich 2020, Luciano 2022, Project Mesa). Each Ant agent follows a set of simple rules and the collective system benefits. The general order of operations is as follows:

1. **Create world:** Initialize the grid world with food locations and a nest location.
2. **Create agents:** Populate Ant agents at the nest location.
3. **Ant run:** Each Ant executes a run. A run consists of the Ant agent visiting every food location and returning home to the nest. Ants decide which food location to visit based on the level of pheromones in the environment.
4. **Pheromones:** When an Ant returns home, pheromones are added to the path they took. The amount of pheromones added is relative to how fast they were.
5. **Repeat:** Once all Ants have returned, repeat steps 3-5.

Step 1: Create world

```
class AntWorld(mesa.Model):  
  
    def __init__(self, num_ants, height=50, width=50, num_runs=3, sig=0.7):  
        super().__init__()  
        # set model variables from inputs  
        self.num_ants = num_ants  
        self.num_runs = num_runs # not yet implemented  
        self.sigma = sig  
        # build the grid, with wrap-around turned off (False)  
        self.grid = mesa.space.MultiGrid(width, height, False)  
        # set schedule  
        self.schedule = mesa.time.SimultaneousActivation(self)  
        self.running = True  
        # additional model-level variables  
        self.ant_scores = []  
        self.score_log = [] # not yet implemented for graphing  
        self.ant_list = []  
        self.reset_ants = 0
```

The inputs for AntWorld are the number of ants, the number of total runs, and the sigma value. The total number of runs input has no bearing on the current program, as I didn't get a chance to implement a cutoff. AntWorld also takes the height and width of the world as inputs, with the default set to 50x50.

Step 2: Create agents

Agents are defined within the `[ant_agents.py]` file. They are populated within the `AntWorld` definition.

Nest population (within `AntWorld` definition, `[ant_model.py]`):

```
# define location of nest / eventually make this random
self.nestloc = (2,2)
# populate nest
self.nest = Nest(self.next_id(), self.nestloc, self)
self.grid.place_agent(self.nest, self.nestloc)
self.schedule.add(self.nest)
```

Food population:

```
# populate food / eventually make these random locations
self.foodlocs = ((10,20), (30,45), (15,40), (46,2))
self.all_food = []
for loc in self.foodlocs:
    food = Food(self.next_id(), self)
    self.grid.place_agent(food, loc)
    self.schedule.add(food)
    self.all_food.append(food)
```

Food and the nest are populated at specific arbitrary locations, which I chose randomly while first setting up `AntWorld`.

Ant population:

```
# populate ants at nest
for i in range(self.num_ants):
    ant = Ant(self.next_id(), self.nest, self.all_food, self)
    self.ant_list.append(ant)
    self.grid.place_agent(ant, self.nestloc)
    self.schedule.add(ant)
```

Environment cell population:

```
# create environment cells, one for each grid cell
for (contents, x, y) in self.grid.coord_iter():
    cell = Environment(self.next_id(), (x, y), self)
    self.grid.place_agent(cell, (x, y))
    self.schedule.add(cell)
```

Step 3: Ant run

From class `Ant(mesa.Agent)` definition within `[ant_agents.py]`

```
def antrun(self):
    # check to see if ant is complete
    if self.pos == self.nest.pos and len(self.food_history) > 0 and
        self.complete is False:
        # if complete, set self.complete to True
        self.complete = True
        # add to Nest's counter
        self.nest.successful_ants += 1
        # add grid history to ant_scores list from the model
        self.model.ant_scores.append(len(self.grid_history))
        # add pheromones to path taken
        self.update_phero()
        return
    elif self.pos == self.nest.pos and self.complete is True:
        return
    else:
        pass
    # if ant is not complete, add current location to grid history
    self.grid_history.append(self.pos)
    # see if there's food on current location
    cellmates = self.model.grid.get_cell_list_contents([self.pos])
    for agent in cellmates:
        if type(agent) is Food:
            # if there is food, add it to the food history
            self.food_history.append(agent.unique_id)
            # choose a new path
            self.choose_path()
        else:
            pass
    # if there's no food on current cell, continue on path
    self.path_move()
```

Because of how this collective system is designed, no learning was done by any of the agents or even the model itself. The system isn't trained, and neither are the Ants. The only element that changed throughout the simulation is the level of pheromone on any given environment cell.

Results and Conclusions:

Unfortunately, I completed the basic simulation just before the end of the semester and was unable to run any kind of experiments or system performance tests. The simulation, as it is now, runs rather slowly, but correctly. Due to this, I'm unable to draw any major conclusions from my work. However, enough literature suggests that an ACO like this could be improved to function highly efficiently at finding good TSP solutions.

Currently, speed is one of the biggest difficulties I've run into. Using Mesa to visualize every single Ant step is slowing down the system. Although Mesa gave me the kind of visualization I was hoping for at the beginning of this project, moving forward I would use a much less complicated GUI— something as simple as graphing with matplotlib.

There are many additional improvements and modifications I'd like to make to the simulation as it stands right now:

- An updating graph of the average Ant score and best Ant score over time should be added to the server as a method of performance evaluation. If the ACO is working correctly, then not only will the two lines decrease over time, but they will also draw closer as Ants decide on one collective path.
- Adding pheromone dispersal to the simulation would make the model more accurate to a real-life ant foraging environment, and could result in better solutions.
- Randomizing food and nest locations is a very important addition to this model. Ideally, one of the model inputs would be the number of nodes. From there the model should populate the food and nest locations randomly, according to the total number of nodes. By populating the food network differently every time, the ACO can be tested to see if it performs better on certain kinds of networks over others.
- Solving the network by brute force to find the shortest path would be a good method of verifying the ACO's chosen paths.
- Once all of these updates and additions are made, the ACO should be tested with varying amounts of ants and food, as well as with different sigma values.

How to Run:

The following files are needed to run AntWorld, and must be in the same folder:

- `[ant_model.py]`
- `[ant_agents.py]`
- `[ant_launcher.py]`

The following packages are needed to run the simulation:

- mesa
- math

- bresenham
- random
- matplotlib
- math

To run the simulation, run `[ant_launcher.py]`. A server should open using your system's default web browser.

Sources:

Bajpai A, Yadav R. 2015. Ant Colony Optimization (ACO) For The Traveling Salesman Problem (TSP) Using Partitioning. 4(09).

Bianchi L, Gambardella LM, Dorigo M. 2002. An Ant Colony Optimization Approach to the Probabilistic Traveling Salesman Problem. In: Guervós JJM, Adamidis P, Beyer H-G, Schwefel H-P, Fernández-Villacañas J-L, editors. Parallel Problem Solving from Nature – PPSN VII. Berlin, Heidelberg: Springer. (Lecture Notes in Computer Science). p. 883–892.

Bioinformatics and other bits - A network agent based infection model with Mesa. [accessed 2023 May 14]. <https://dmnfarrell.github.io/bioinformatics/abm-mesa-network>.

Bresenham's Line Generation Algorithm. 2017. GeeksforGeeks. [accessed 2023 May 14]. <https://www.geeksforgeeks.org/bresenhams-line-generation-algorithm/>.

Chen S-M, Chien C-Y. 2011. Solving the traveling salesman problem based on the genetic simulated annealing ant colony system with particle swarm optimization techniques. Expert Systems with Applications. 38(12):14439–14450. doi:[10.1016/j.eswa.2011.04.163](https://doi.org/10.1016/j.eswa.2011.04.163).

Deneubourg J-L, Goss S, Franks N, Pasteels J. 1989. The blind leading the blind: Modeling chemically mediated army ant raid patterns. Journal of Insect Behavior. 2:719–725. doi:[10.1007/BF01065789](https://doi.org/10.1007/BF01065789).

Dorigo M, Birattari M, Stutzle T. 2006. Ant colony optimization. IEEE Comput Intell Mag. 1(4):28–39. doi:[10.1109/MCI.2006.329691](https://doi.org/10.1109/MCI.2006.329691).

Dorigo M, Gambardella LM. 1997a. Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans Evol Computat. 1(1):53–66. doi:[10.1109/4235.585892](https://doi.org/10.1109/4235.585892).

Dorigo M, Gambardella LM. 1997b. Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans Evol Computat. 1(1):53–66. doi:[10.1109/4235.585892](https://doi.org/10.1109/4235.585892).

Induraj. 2023. Implementing Ant colony optimization in python- solving Traveling salesman problem. Medium. [accessed 2023 May 14].
<https://induraj2020.medium.com/implementation-of-ant-colony-optimization-using-python-solve-traveling-salesman-problem-9c14d3114475>.

Jun-man K, Yi Z. 2012. Application of an Improved Ant Colony Optimization on Generalized Traveling Salesman Problem. Energy Procedia. 17:319–325. doi:[10.1016/j.egypro.2012.02.101](https://doi.org/10.1016/j.egypro.2012.02.101).

Kazil J, Masad D, Crooks A. 2020. Utilizing Python for Agent-Based Modeling: The Mesa Framework. In: Thomson R, Bisgin H, Dancy C, Hyder A, Hussain M, editors. Social, Cultural, and Behavioral Modeling. Cham: Springer International Publishing. (Lecture Notes in Computer Science). p. 308–317.

Kuo C. 2022. Building An Agent-Based Model with Python for Ant Foraging. MLearning.ai. [accessed 2023 May 14].
<https://medium.com/mllearning-ai/building-an-agent-based-model-with-python-streamlit-for-ant-foraging-a74a41605d2c>.

Manfrin M, Birattari M, Stützle T, Dorigo M. 1970. Parallel Ant Colony Optimization for the Traveling Salesman Problem. p. 234.

McGuigan J. Ant Colony Optimization Algorithm. Kaggle. [accessed 2023 May 14].
<https://kaggle.com/code/jamesmcguigan/ant-colony-optimization-algorithm>.

Shukla P. 2022. Ant Colony Optimization...Genetic Programming & Pygame. Neoned71. [accessed 2023 May 14].
<https://neoned71.com/ant-colony-optimization-genetic-programming-pygame/>.

Waldner J-B. 2008. Nanocomputers and Swarm Intelligence.

Github repositories:

https://github.com/projectmesa/mesa-examples/blob/main/examples/virus_on_network/virus_on_network/server.py

Goadrich, M. 2020. Ants Model: A port of the ants simulation to Mesa agent-based modeling framework in Python. San Francisco (CA): GitHub; [accessed 2023 April 28]. <https://github.com/mgoadric/ants-mesa>.

Luciano. 2022. Ant Colony Optimization: Implementation from the ground up in Python of the Ant Colony Optimization algorithm for Traveling Salesman Problem. San Francisco (CA): GitHub; [accessed 2023 May 2]. <https://github.com/StrikingLoo/ant-colony-optimization>

Project Mesa. Mesa Examples. San Francisco (CA): GitHub; [accessed 2023 April 28]. <https://github.com/projectmesa/mesa-examples>